

Design and Analysis of Algorithms

Lecture 6: Divide and Conquer Algorithms (Part 2)





Table of Contents for this topic

- Divide and Conquer Approach
- Searching: binary search
- Finding maximum and minimum
- Powering a number
- Sorting: merge sort and quicksort
- Matrix multiplication: Strassen's algorithm





Agenda for this lecture

- Sorting: merge sort and quicksort
- Matrix multiplication: Strassen's algorithm



Example (4): Given an array of n elements, the goal is to rearrange the elements into ascending (or descending) order.

The Naïve Algorithm:
Bubble Sort

Divide-and-conquer
algorithm: Merge Sort

Divide-and-conquer
algorithm: Quick Sort

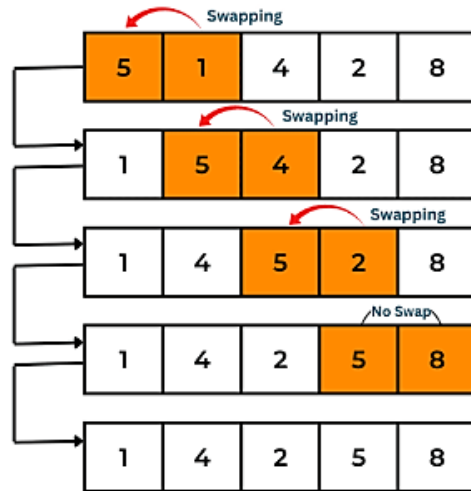
Bubble Sort (1/3)

- Bubble Sort is a simple comparison-based algorithm. It repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order.
- The pass through the list is repeated until no swaps are needed, indicating the list is sorted. The algorithm gets its name because smaller elements "bubble" to the top of the list.

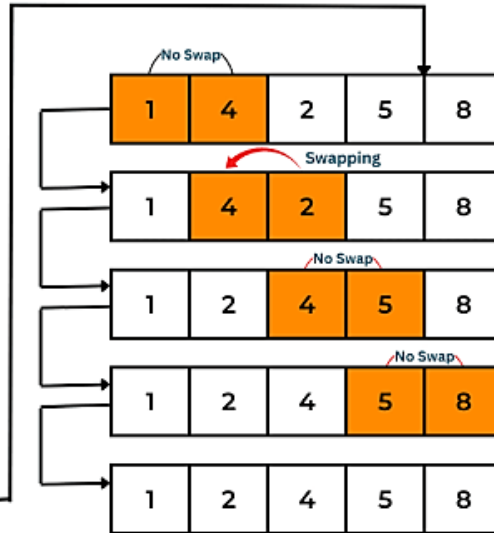


Bubble Sort (2/3)

First Pass



Second Pass



Third Pass



Bubble Sort (3/3)

```
FUNCTION bubbleSort(A)
  // A: the array to be sorted
  n = length(A)
  FOR i FROM 0 TO n-2 DO
    swapped = false
    FOR j FROM 0 TO n-i-2 DO //Last i elements are already in place
      IF A[j] > A[j+1] THEN
        SWAP(A[j], A[j+1])
        swapped = true
      END IF
    END FOR
    // If no two elements were swapped, the array is sorted.
    IF NOT swapped THEN
      BREAK
    END IF
  END FOR
END FUNCTION
```

● Time Complexity: $O(n^2)$

- Worst Case: $O(n^2)$ - The array is in reverse order. We need n passes, each doing $\sim n$ comparisons.
- Best Case: $O(n)$ - The array is already sorted. The optimized version (with swapped flag) will only do one pass.
- Average Case: $O(n^2)$

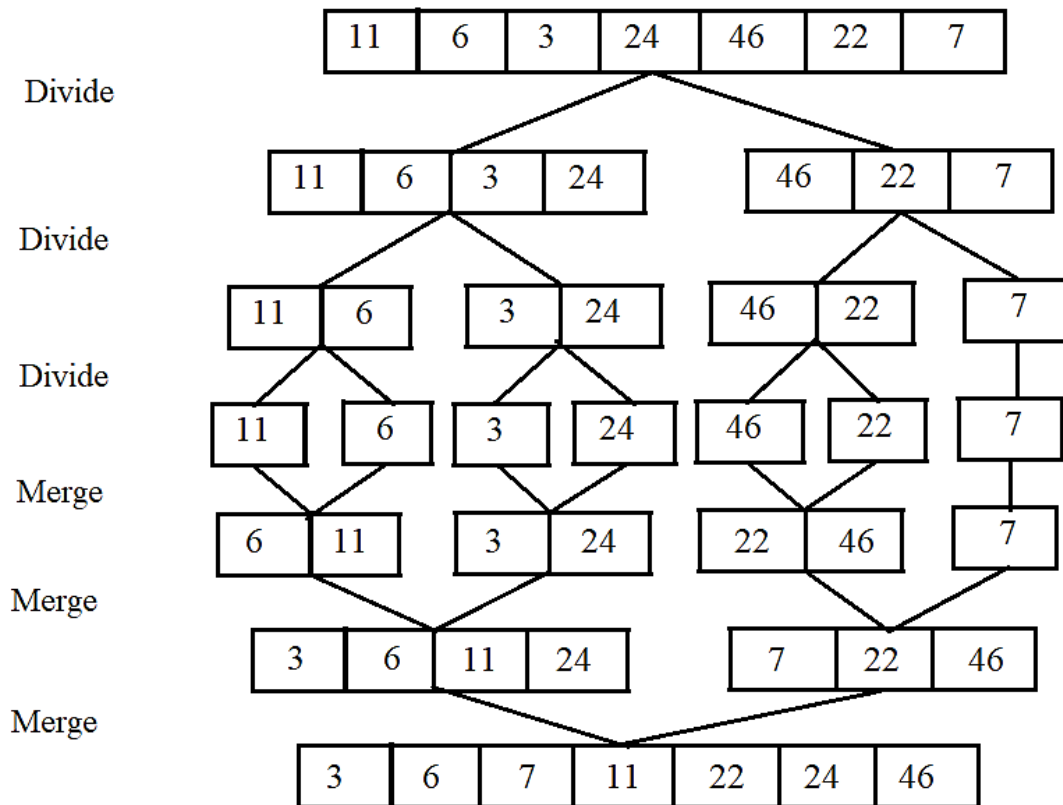


Merge Sort (1/5)

- **Merge Sort:** is a classic **divide-and-conquer algorithm**.
- It works by recursively dividing the array into smaller subarrays until each subarray contains a single element (which is trivially sorted).
- Then, it repeatedly merges the subarrays to produce new sorted subarrays until there is only one sorted array remaining.



Merge Sort (2/5)



Merge Sort (3/5)

- Recurrence Relation: $T(n) = 2T(\frac{n}{2}) + n$
 - ✓ $2T(\frac{n}{2})$: Time to recursively sort two subarrays of size $n/2$.
 - ✓ n Time to merge the two sorted subarrays.
- Time Complexity: $(n \log n)$ in all cases (worst, average, best), calculated by master method.

```
def merge_sort(A, low, high):  
    # Base case: a subarray of length 1 or 0 is already sorted  
    if low >= high:  
        return  
  
    # Recursive case: divide the array into two halves  
    mid = (low + high) // 2  
  
    # Recursively sort the left half  
    merge_sort(A, low, mid)  
  
    # Recursively sort the right half  
    merge_sort(A, mid + 1, high)  
  
    # Merge the two sorted halves  
    merge(A, low, mid, high)  
  
def merge(A, low, mid, high):  
    # Create temporary arrays for left and right halves  
    left = A[low:mid+1]  
    right = A[mid+1:high+1]  
  
    i = j = 0  
    k = low  
  
    # Merge the two arrays by comparing elements  
    while i < len(left) and j < len(right):  
        if left[i] <= right[j]:  
            A[k] = left[i]  
            i += 1  
        else:  
            A[k] = right[j]  
            j += 1  
        k += 1  
  
    # Copy any remaining elements from left array  
    while i < len(left):  
        A[k] = left[i]  
        i += 1  
        k += 1  
  
    # Copy any remaining elements from right array  
    while j < len(right):  
        A[k] = right[j]  
        j += 1  
        k += 1
```

Merge Sort (5/5)

Complete Recursive Call Trace for Merge Sort:

Array: [8, 3, 5, 1]

```
merge_sort([8,3,5,1], 0, 3) called
├─ Line 2: if 0 >= 3? → False
├─ Line 6: mid = (0+3)//2 = 1
├─ Line 9: merge_sort([8,3,5,1], 0, 1) called
│   ├─ Line 2: if 0 >= 1? → False
│   ├─ Line 6: mid = (0+1)//2 = 0
│   ├─ Line 9: merge_sort([8,3,5,1], 0, 0) called
│   │   ├─ Line 2: if 0 >= 0? → True (BASE CASE)
│   │   └─ Line 3: return
│   └─ Line 12: merge_sort([8,3,5,1], 1, 1) called
│       ├─ Line 2: if 1 >= 1? → True (BASE CASE)
│       └─ Line 3: return
└─ Line 15: merge([8,3,5,1], 0, 0, 1) executed
    ├─ left = [8], right = [3]
    ├─ i=0, j=0, k=0
    ├─ Compare: 8 <= 3? → False → A[0] = 3, j=1, k=1
    ├─ Copy: A[1] = 8, i=1, k=2
    └─ Array: [3,8,5,1]
```

Merge Sort (5/5)

Complete Recursive Call Trace for Merge Sort:

Array: [8, 3, 5, 1]

```
└─ Line 12: merge_sort([3,8,5,1], 2, 3) called
    │   └─ Line 2: if 2 >= 3? → False
    │   └─ Line 6: mid = (2+3)//2 = 2
    │   └─ Line 9: merge_sort([3,8,5,1], 2, 2) called
    │       │   └─ Line 2: if 2 >= 2? → True (BASE CASE)
    │       │   └─ Line 3: return
    │   └─ Line 12: merge_sort([3,8,5,1], 3, 3) called
    │       │   └─ Line 2: if 3 >= 3? → True (BASE CASE)
    │       │   └─ Line 3: return
    │   └─ Line 15: merge([3,8,5,1], 2, 2, 3) executed
    │       │   └─ left = [5], right = [1]
    │       │   └─ i=0, j=0, k=2
    │       │   └─ Compare: 5 <= 1? → False → A[2] = 1, j=1, k=3
    │       │   └─ Copy: A[3] = 5, i=1, k=4
    │       └─ Array: [3,8,1,5]
└─ Line 15: merge([3,8,1,5], 0, 1, 3) executed
    │   └─ left = [3,8], right = [1,5]
    │   └─ i=0, j=0, k=0
    │   └─ Compare: 3 <= 1? → False → A[0] = 1, j=1, k=1
    │   └─ Compare: 3 <= 5? → True → A[1] = 3, i=1, k=2
    │   └─ Compare: 8 <= 5? → False → A[2] = 5, j=2, k=3
    │   └─ Copy: A[3] = 8, i=2, k=4
    └─ Array: [1,3,5,8]
```

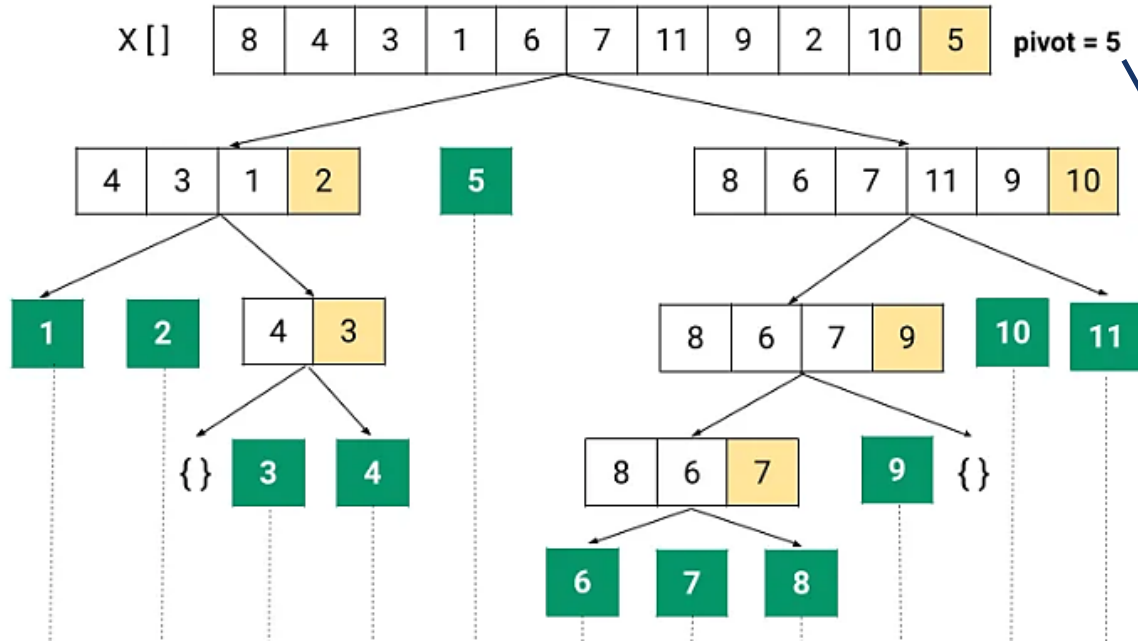
FINAL SORTED ARRAY: [1, 3, 5, 8]

Quick Sort (1/5)

- Quick Sort is also a divide-and-conquer algorithm.
- It works by selecting a 'pivot' element and partitioning the array around the pivot such that all elements smaller than the pivot come before it, and all elements greater come after it.
- The algorithm then recursively sorts the subarrays on either side of the pivot.



Quick Sort (2/5)



Pivot can be the last element

Pivot can be the middle element, first element, or random element



Quick Sort (3/5)

Here's the code for quicksort:

```
def quicksort(array):  
    if len(array) < 2:  
        return array ← Base case: arrays with 0 or 1 element are already "sorted."  
    else:  
        pivot = array[0] ← Recursive case  
        less = [i for i in array[1:] if i <= pivot] ← Sub-array of all the elements  
                                                    less than the pivot  
        greater = [i for i in array[1:] if i > pivot] ← Sub-array of all the elements  
                                                    greater than the pivot  
        return quicksort(less) + [pivot] + quicksort(greater)  
  
print quicksort([10, 5, 2, 3])
```



Quick Sort (4/5)

- **Best Case Recurrence:** $T(n) = 2T(\frac{n}{2}) + n$

Occurs when the pivot always divides the array into two equal halves

- **Worst Case Recurrence:** $T(n) = T(n - 1) + n$

Occurs when the **pivot is always the smallest or largest element**

- **Time Complexity:**

- ✓ Best Case: $\Omega(n \log n)$,
- ✓ Average Case: $\Theta(n \log n)$
- ✓ Worst Case: $O(n^2)$



Quick Sort (5/5)

Complete Recursive Call Trace for Quick Sort:

Array: [10, 5, 2, 3]

```
quicksort([10, 5, 2, 3])
```

```
    pivot = 10
```

```
    less = [5, 2, 3], greater = []
```

```
    return quicksort([5, 2, 3]) + [10] + []
```

```
quicksort([5, 2, 3])
```

```
    pivot = 5
```

```
    less = [2, 3], greater = []
```

```
    return quicksort([2, 3]) + [5] + []
```

```
quicksort([2, 3])
```

```
    pivot = 2
```

```
    less = [], greater = [3]
```

```
    return [] + [2] + quicksort([3])
```

```
quicksort([3]) = [3] // base case
```

Bubble Sort vs. Merge Sort vs. Quick Sort

Input Size (n)	Bubble Sort ($O(n^2)$)	Merge Sort ($O(n \log n)$)	Quick Sort Avg ($O(n \log n)$)	Quick Sort Worst ($O(n^2)$)
n = 10	~100 operations	~33 operations	~33 operations	~100 operations
n = 100	~10,000 ops	~664 ops	~664 ops	~10,000 ops
n = 1,000	~1,000,000 ops	~9,966 ops	~9,966 ops	~1,000,000 ops
n = 10,000	~100,000,000 ops	~132,877 ops	~132,877 ops	~100,000,000 ops
n = 100,000	~ 10^{10} ops	~ 1.66×10^6 ops	~ 1.66×10^6 ops	~ 10^{10} ops



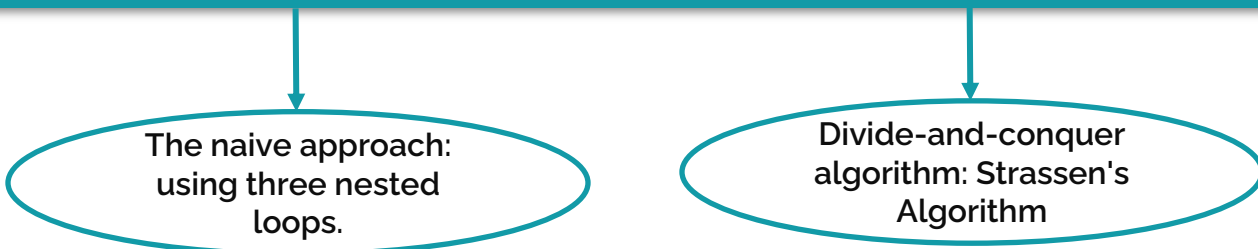
Bubble Sort vs. Merge Sort vs. Quick Sort

- **For problem 4:**

- ✓ Quick Sort's memory efficiency makes it the better choice for most practical in-memory sorting scenarios.
- ✓ The $O(n^2)$ worst case is rare with proper pivot selection.
- ✓ While Merge Sort's memory overhead is always present and can be prohibitive with large datasets.



Example (5): Given two $n \times n$ matrices A and B , compute their product $C = A \times B$, where each element c_{ij} of the resulting matrix C is computed as the dot product of the i -th row of A and the j -th column of B .



The naive approach:
using three nested
loops.

Divide-and-conquer
algorithm: Strassen's
Algorithm



The naive approach for matrix multiplication

- The naive approach directly implements the mathematical definition of matrix multiplication using three nested loops.
- Time Complexity: $O(n^3)$.

```
FUNCTION naiveMatrixMultiply(A, B, n)
    // A, B: input matrices of size n × n
    // n: dimension of matrices

    LET C be a new n × n matrix initialized to zeros

    FOR i FROM 0 TO n-1 DO
        FOR j FROM 0 TO n-1 DO
            FOR k FROM 0 TO n-1 DO
                C[i][j] = C[i][j] + A[i][k] * B[k][j]
            END FOR
        END FOR
    END FOR

    RETURN C
END FUNCTION
```

Divide and Conquer: Strassen's Algorithm

- Strassen's algorithm is a "Divide and Conquer" algorithm for matrix multiplication.
- It uses a clever algebraic method to compute the result with 7 recursive.
- reduces the time complexity from $O(n^3)$ to $O(n^{\log_2 7}) \approx O(n^{2.81})$.



Divide and Conquer: Strassen's Algorithm

The implementation of this algorithms is your next assignment



Any Question?

